

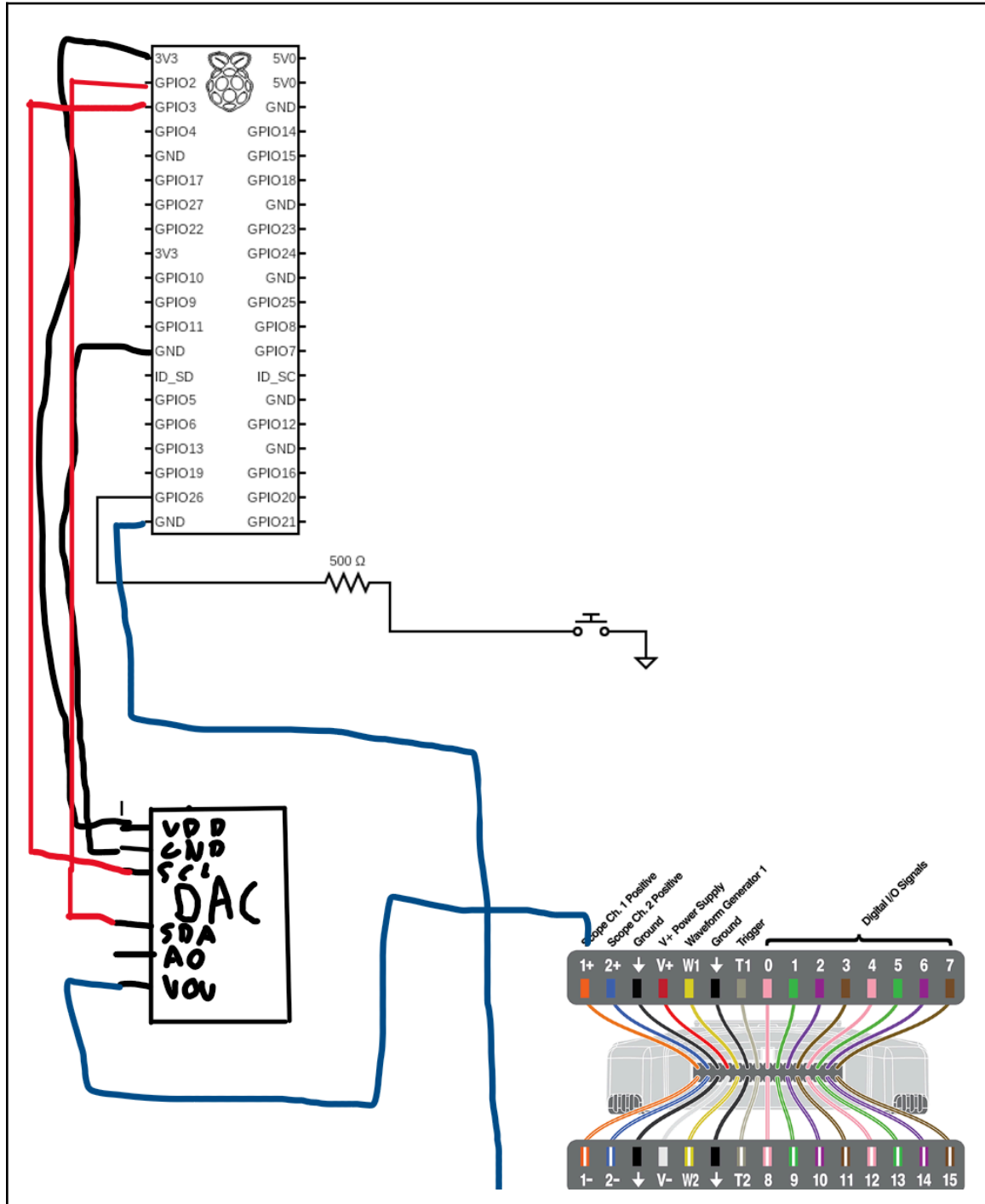
Lab 2: Use of Digital to Analog Converter along with a Microcontroller

Katelyn Nguyen
Oluwasegun Oluwole
CSCE 462 - 500
Group 1

06 September 2024

Hardware Description

Provide a written description of your hardware **including a wiring diagram**. Wiring diagrams can be screenshots from [multisim](https://www.multisim.com/) (online or desktop applications), circuit.io, or a **LEGIBLE** photo of a hand-drawn diagram.





Note: Pin to Pin connections are all made via jump wires

The hardware components used in this lab consist of a Raspberry Pi (model 3b+) , jump wires, an Analog Discovery 2 Device, a DAC, and a push button. The 3V3 pin from the raspberry pi extension kit is connected to the VDD for the DAC, supplying power to it and one of the GND pins from the raspberry pi extension kit is connected to the GND of the DAC. SDA1 which refers to GPIO2 on the raspberry pi extension kit is connected to the SDA pin on the DAC and sends data from the raspberry pi to the chip. SCL1 which refers to GPIO3 on the raspberry pi extension kit is connected to the SCL pin on the DAC and this controls the output rate. GPIO26 will act as an input and is connected to a 500 ohm resistor which is connected to a push button. Channel 1 on the AD2 is used to observe the waveform output. This is done by connecting the VOU pin on the DAC to the positive terminal of channel 2 and then connecting the negative terminal of channel 2 to GND.

Software Description

Provide a description of your code and its implementation. **Remember to include your code in your submissions as a separate file. DO NOT PROVIDE A ZIP, TAR, or other file format as your submission unless specified.**

```
import time

import RPi.GPIO as GPIO # type: ignore

import board # type: ignore
import busio # type: ignore
import math

import adafruit_mcp4725 # type: ignore

# Initialize I2C bus.
i2c = busio.I2C(board.SCL, board.SDA)

# Initialize MCP4725.
dac = adafruit_mcp4725.MCP4725(i2c)
```

This is the beginning of the code and this includes the necessary library imports for this

lab as well as initialization of the I2C Bus and the DAC.

```
button = 26

GPIO.setmode(GPIO.BCM)
GPIO.setup(button, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

This block of code sets the push button as an input to the GPIO26 pin on the raspberry pi. In addition, the initial state of the button is high as a pull up resistor is used in the code to pull its value up to high.

```
def stop_wave():
    dac.raw_value = int(0)
    GPIO.output(outPin, GPIO.LOW)

def button_logic():
    if GPIO.input(button) == GPIO.LOW:
        stop_wave()
        waveform_type = str(input("Choose a waveform type
(sq/sin/tri): "))
        while True:
            if (waveform_type == "sq" or waveform_type == "sin" or
waveform_type == "tri"):
                inputVmax = float(input("What is your desired
maximum voltage? (Vmax): "))
                inputFreq = float(input("What is your desired
frequency? (freq): "))
                if (waveform_type == "sq"):
                    print("Outputting Square Wave")
                    print("Press the button again to start a new
waveform")
                    square_wave(inputVmax, inputFreq)
                elif waveform_type == "sin":
                    print("Outputting Sine Wave")
                    print("Press the button again to start a new
waveform")
                    sin_wave(inputVmax, inputFreq)
                elif waveform_type == "tri":
                    print("Outputting Triangle Wave")
                    print("Press the button again to start a new
```

```

waveform")
        triangle_wave(inputVmax, inputFreq)
    else:
        print("Invalid input, please try again.")
    else:
        # print(waveform_type, type(waveform_type))
        print("Invalid input, please try again.")
else:
    pass

```

This block of code includes the stop_wave function which sets the dac.raw_value to 0. The button_logic contains the main execution sequence of the system which waits till the button is pressed and then prompts the user for which wave they want outputted and then the desired maximum voltage and maximum frequency. In button_logic() the program continually checks the value of the button(GPIO26) via polling.

```

def square_wave(Vmax, freq):
    # Uses the DAC to convert the function inputs to obtain a
    waveform via the AD2

    # function inputs:
    # freq is the desired frequency in Hz
    # Vmax is the maximum desired output voltage in Volts

    t = 0.0 # starting time

    # tStep = 0.05 # time steps # no longer in use

    while True:
        start = time.time()
        # tStep is based on execution time

        if ((t%float(1/freq)) > (float(1/freq)/2)):
            # "1/freq" is the period
            # if the amount of time modulo the period is greater
            than half the period

            dac.raw_value = int((4095 * Vmax)/3.3)
            # 4095 is the raw 12-bit output of the DAC
            # dac.raw_value is for the Vmax in bits (not

```

```

Volts)

        # 3.3 Volts is the input voltage from our
raspberry pi

        # this is converting the inputted Vmax into bits,
and setting it in the DAC

    else:
        dac.raw_value = 0

    t += time.time() - start
    # this is the new tStep

    button_logic()

```

The above code block contains the function for the square wave, which takes in an input Vmax and a frequency. The main logic in the while loop checks if at the current time, the time is greater than half of the period which is 1 over the frequency the output will be Vmax and else the output will be 0 and then button_logic() is at the end of the while loop to which will stop the waveform output and prompt the user again.

```

def sin_wave(Vmax, freq):
    t = 0.0

    tStep = 0.05

    Vmax_raw_value = (Vmax*4095)/3.3

    while True:
        start = time.time()

        #THIS VARIABLE voltage IS IN BITS:
        voltage = (Vmax_raw_value/2) + (Vmax_raw_value) * (0.5*
math.sin((6.2832*freq)*t)) # increasing what's inside of sin,
increases the frequency

        dac.raw_value = int((voltage))

        t+=time.time()-start
        # t+=tStep

```

```
# time.sleep(0.0005)
```

```
button_logic()
```

The above code block contains the sine wave function which takes as input, a Vmax and a frequency. The Vmax_raw_value is basically a proportion we used to convert the inputted Vmax to its corresponding 12 bit raw value. The sin wave is compressed by a factor of 2 and shifted up by factor of the (Vmax_raw_value divided by 2) and then button_logic() is called again which will stop the waveform output and enter the main execution loop of the system.

```
def triangle_wave(Vmax, freq):  
    # Input Vmax in Volts and freq in Hz  
  
    t = 0.0  
    # tStep = 0.05 # no longer in use  
  
    Vmax_raw_value = int((4095*Vmax)/3.3)  
  
    while True:  
        start = time.time()  
        if (t == 0):  
            dac.raw_value = 0  
            elif ((t*float(1/freq)) < (float(1/freq)*0.5)):  
                dac.raw_value =  
int((Vmax_raw_value*2*(freq))*(t%(1/freq)))  
                # equation for the line: 2Vmax*T * (t % T);  
T=period=(1/freq)  
            else:  
                dac.raw_value =  
int((( -1)*Vmax_raw_value*2*(freq))*(t%(1/freq))+(2*Vmax_raw_value)  
)  
                # equation for the line: -2Vmax*T * (t % T) + 2Vmax;  
T=period=(1/freq); adding 2Vmax for y-int for negative slope  
  
                t += time.time() - start  
                # this is the new tStep  
  
        button_logic()
```

The above code block contains the code for the triangle wave which takes in an input

Vmax and frequency and the Vmax is converted via the the same proportion used in the previous sine wave function. If the current time of execution is 0 then the raw value is 0. If the time is less then half the period, this is the upward part of the triangle wave and the dac.raw_value will be the Vmax_raw_value times $2 * \text{frequency} * \text{current time}$. Otherwise, it will be the downward part of the triangle wave and the dac.raw_value will equal $-1 * \text{Vmax_raw_value} * 2 * \text{frequency} * \text{current time} + (2 * \text{Vmax_raw_value})$. The $(2 * \text{Vmax_raw_value})$ is to ensure that the peak of the triangle wave is on the same line on both sides of the wave. Then button_logic() is called again to stop the waveform output and resume the main execution logic of the system.

```
def main():
    # print("Howdy! Welcome to Segun's and Katelyn's Lab 2
    Oscilloscope!")
    # print("Press the button to begin.")
    try:
        while True:
            time.sleep(0.1)
            button_logic()

    finally:
        stop_wave()
        GPIO.cleanup()

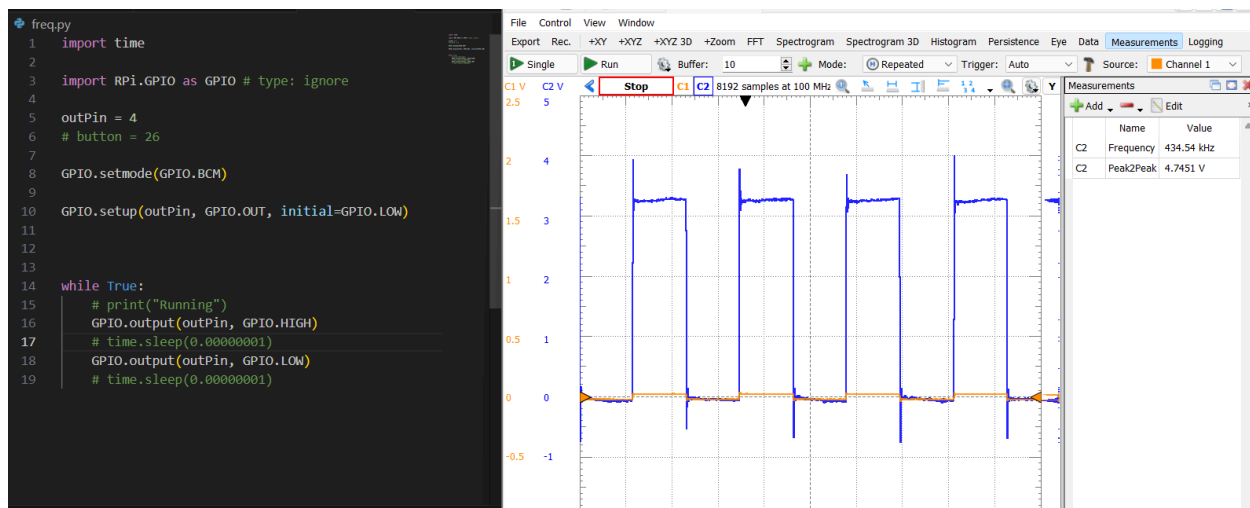
if __name__ == "__main__":
    main()
```

The above code block contains the main function of the system. There is a while loop which runs button_logic() and at the end stop_wave() is called and GPIO.cleanup() is called to clean up all values on the GPIO pins for their next use.

Lab Questions

Provide the **questions and answers** to the questions included in the lab manual.

1. What is the highest frequency that your Raspberry Pi (model 3b+) can generate using digital output? Why is this the case?
 - a. 445 kHz



In the above code, we chose an output pin and just toggled between high and low without having time.sleep() to see what the highest frequency is that the raspberry pi generates.

2. Does the signal with the highest frequency stay steady or fluctuate? If not and there is noise, where does the noise come from?
 - a. The signal with the highest frequency does fluctuate and this could be due to the raspberry pi attempting to overclock which will result in noise because the signal did not have enough time to steady out.
3. How can you convert a digital PWM (Pulse width modulation) signal into an analog signal? (e.g. possible circuit design, software conversion)
 - a. A digital PWM signal can be converted into an analog signal by using a low pass filter which will smooth out the signal by removing all the high-frequency components of the signal above a certain threshold. This can also be done via digital signal processing.
4. What is the maximum frequency you can produce with your Raspberry Pi functional generator on different wave shapes?
 - a. The maximum frequency the functional generator can produce for the square wave is 790 Hz. The maximum frequency the functional generator can produce for the triangle wave is 200 Hz. The maximum frequency the functional generator can produce for the sine wave is 200Hz. We came to this conclusion because after increasing the frequency higher than the aforementioned values, the shapes were not distinct.
5. Explain your design for the functional generator (you can use diagrams to visualize your system state machine, what function you implemented, etc.)
 - a. We utilized the polling method for checking the if the state of the button had gone from high to low. If low then the main execution of the system would occur: The user is prompted to enter the type of wave they want(square, triangle, or sine) displayed, the desired maximum output voltage from 0 to Vcc, and the desired frequency from 0 Hz to 50 Hz. Then the output of the wave is displayed. Once the button is pressed again, the waveform is no longer outputted and the user is prompted

again to enter the three parameters for generating the waveform. The functions for calculating and displaying each of the waveforms is given in the above software description as well as in the code that will be attached with this report.

